1. Project Overview

You are building the backbone for a high-demand concert ticketing platform. Unlike a basic CRUD app, this system must survive thousands of simultaneous clicks for a limited number of seats. You must ensure that no two people can buy the same seat, and the system must remain fast even as the database grows.

2. Technical Stack

- **Runtime:** Node.js with TypeScript
- **Framework:** Express.js
- **ORM:** TypeORM
- **Database:** SQLite (file-based)
- **Tools:** AI assistance (Cursor, Copilot, etc.) is allowed, but architecture must be manually verified. (Just don't drop this whole pdf file to AI xD)

3. Functional Requirements

Your API must implement the following:

1. **Concert Discovery:** GET /concerts — List all concerts and their current available stock.
2. **Atomic Reservation:** POST /reserve — Reserve 1 ticket for a userId for 5 minutes.
    - *Rule:* If stock is 0, the request must fail.
    - *Rule:* If the reservation logic fails halfway, stock must not be lost.
3. **Purchase Confirmation:** POST /purchase — Convert a PENDING reservation to COMPLETED.
4. **Cleanup Logic:** cleanup cron job — A manual or automated trigger to release all PENDING reservations that have expired (older than 5 mins) back into available stock.

4. Non-Functional Requirements

A. Schema Integrity & Migrations

- **Constraint:** Set synchronize: false in your TypeORM config.
- **Task:** You must create your initial tables using migration:generate.
- **Change Request:** Halfway through, you must add a category column (e.g., 'VIP', 'General') to the Ticket table using a **second migration**.

B. The Performance Layer (Indexing)

- **Task:** Implement a **B-Tree Index** on the concertId column.
- **Task:** Implement a **Partial Index** on the status column specifically for PENDING records. (Research why we do this)
- **Question:** In your README, explain why a Partial Index is better than a standard index for the "cleanup" task.

C. Concurrency Control (ACID)

- **Task:** Use queryRunner.startTransaction() for the reservation flow.
- **Proof:** You must write a test or log that proves if the Reservation record fails to save, the Product stock is **rolled back**.

5. Submission Deliverables

1. **Source Code:** A GitHub link containing your TypeScript files.
2. **The Migration Folder:** Must contain at least two files showing schema evolution.
3. **The "Explain" Output:** Use EXPLAIN QUERY PLAN (the SQLite version of Explain Analyze) to prove your index is working for the search query.
4. **The README:** A short summary of:
    - How you handled the "Double-Selling" problem.
    - Why you chose the specific columns for indexing.
    - How "Vibe Coding" (AI) helped or hindered your architectural decisions.